Lógica para Engenharia de Software

Grafos Completos, Bipartidos e Árvores Binárias

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software

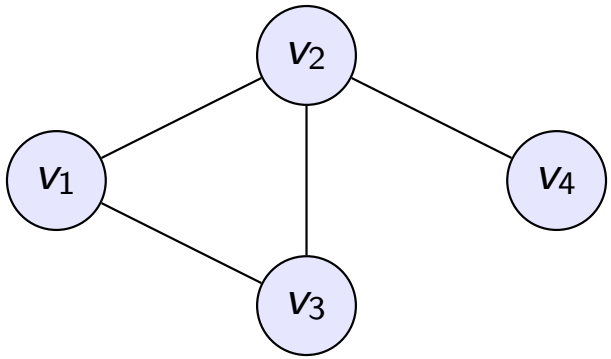


Sumário

1. Introdução aos Grafos Completos
2. Teoremas, Propriedades e Corolários
3. Extensão: Grafos Bipartidos
4. Estruturas Hierárquicas: Árvores Binárias
5. Teoremas e Propriedades das Árvores Binárias

1. O que é um Grafo? (Revisão)

- Um **grafo** $G = (V, E)$ consiste em um conjunto de vértices (V) e um conjunto de arestas (E).
- Vértices representam entidades (nós, pontos).
- Arestas representam relações ou conexões entre essas entidades.
- Na *Lógica para Software*, grafos são estruturas fundamentais para modelar redes, dependências e fluxo de controle.



2. Definição de Grafo Completo

Definição

Um **grafo completo** é um grafo simples (sem laços ou arestas múltiplas) no qual **todo par de vértices distintos é conectado por uma única aresta**.

- Isso significa que existe uma ligação direta entre todos os elementos do conjunto.
- É o grafo "mais denso" possível para um determinado número de vértices.
- Representa um cenário onde "todo mundo conhece todo mundo" ou "todos os nós estão interligados".

3. Notação

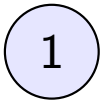
- Um grafo completo com n vértices é denotado pela letra maiúscula K_n.
- O "K" é frequentemente associado ao termo alemão *komplett* ou em homenagem ao matemático Kazimierz Kuratowski.
- Para especificar um grafo completo, basta saber o número de vértices n.

Importante

Um subgrafo de qualquer grafo que por si só forma um grafo completo é chamado de **clique**.

4. Exemplos Iniciais: K_1, K_2 e K_3

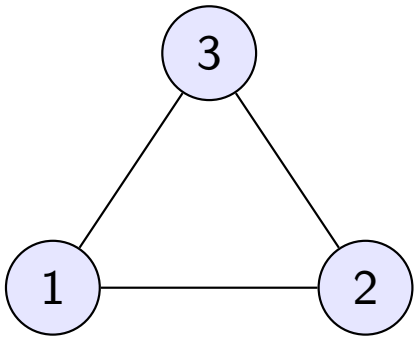
K_1
(1 vértice, 0 arestas)



K_2
(2 vértices, 1 aresta)



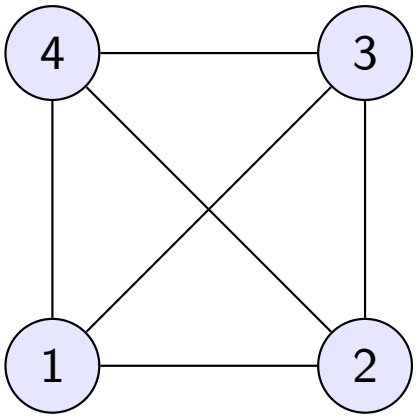
K_3
(3 vértices, 3 arestas)



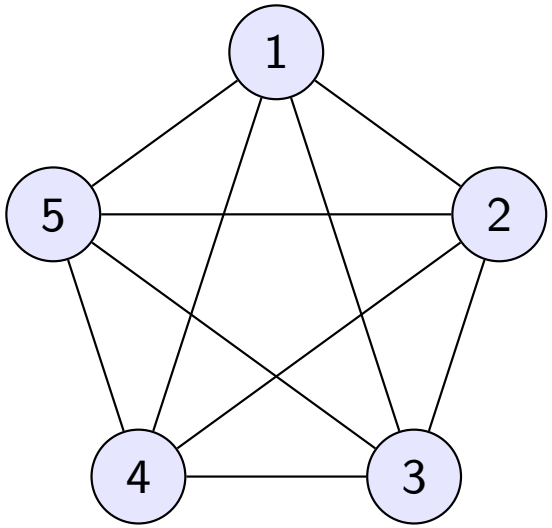
(Um triângulo)

5. Exemplos Maiores: K_4 e K_5

K_4
(4 vértices, 6 arestas)

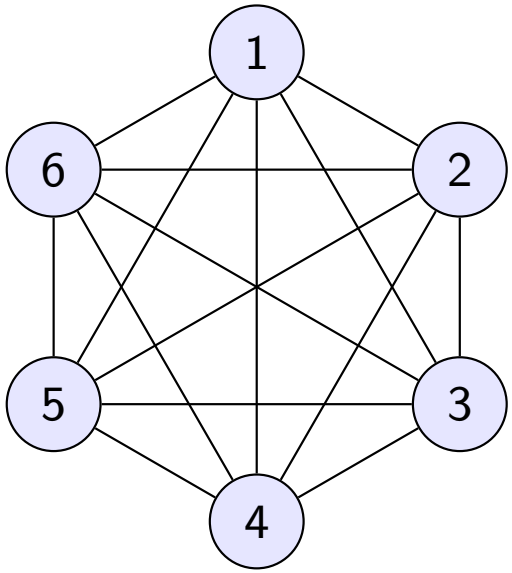


K_5
(5 vértices, 10 arestas)

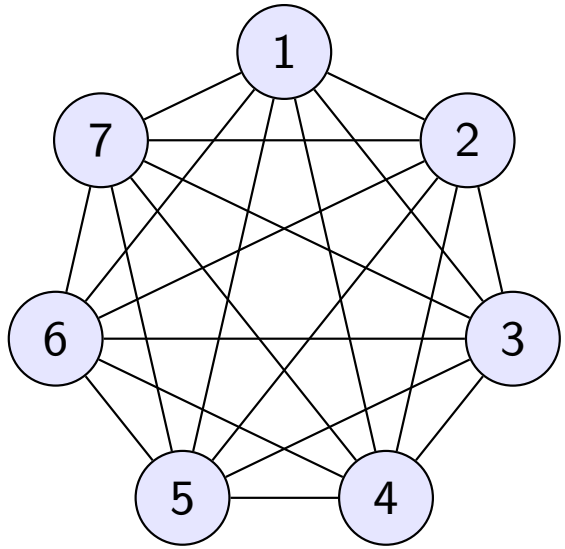


5.1. Mais Exemplos: K_6 e K_7

K_6
(6 vértices, 15 arestas)

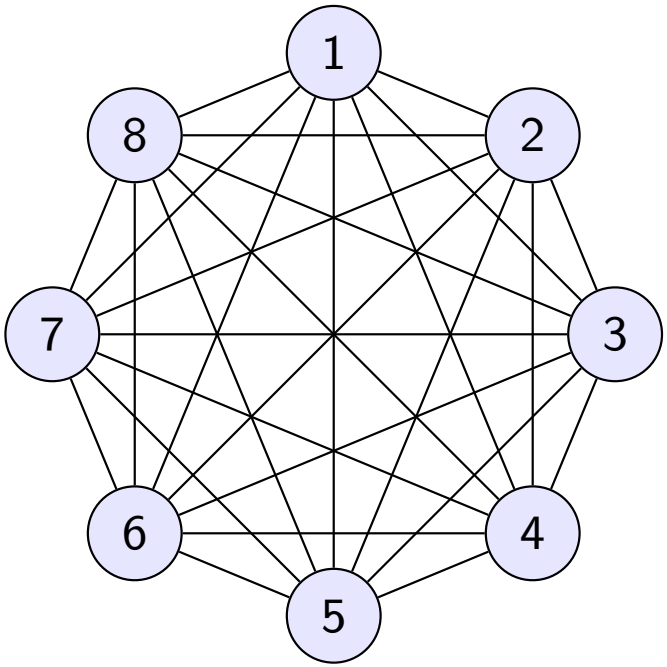


K_7
(7 vértices, 21 arestas)



5.2. Exemplos Extremos: K_8

K_8
(8 vértices, 28 arestas)



- À medida que n cresce, o número de arestas cresce quadraticamente.
- A densidade visual de K_8 ilustra a complexidade máxima (pior caso) de um problema envolvendo 8 entidades totalmente conectadas.
- A representação começa a ficar visualmente densa e embaraçada, reforçando o custo do $O(n^2)$.

6. Propriedades: Número de Arestas

- Como cada vértice se conecta a todos os outros vértices, o número de arestas cresce rapidamente com o número de vértices.
- O número de arestas em um grafo completo K_n é dado pela combinação de n elementos tomados 2 a 2:

Fórmula do Número de Arestas

$$|E| = \binom{n}{2} = \frac{n(n-1)}{2}$$

- Exemplo para K_5: $n = 5 \Rightarrow \frac{5 \cdot 4}{2} = 10$ arestas.
- Exemplo para K_{10}: $n = 10 \Rightarrow \frac{10 \cdot 9}{2} = 45$ arestas.

7. Propriedades: Grau dos Vértices

- O **grau** de um vértice é o número de arestas conectadas a ele.
- Em um grafo completo K_n, todos os vértices são simétricos e se conectam a todos os outros $n-1$ vértices do grafo.

Grau em K_n

Em um grafo completo K_n, o grau de todo vértice é exatamente $n-1$.

- Como todos os vértices têm o mesmo grau, dizemos que K_n é um **grafo regular** de grau $n-1$ (ou simplesmente $(n-1)$-regular).

8. Representação Computacional (Matriz de Adjacência)

- Em computação, grafos são frequentemente representados por Matrizes de Adjacência.
- Para um grafo completo K_n, a matriz de adjacência $A_{n\times n}$ possui zeros na diagonal principal e uns em todas as outras posições.

Exemplo para K_4:

$$A_{K_4} = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

- Isso reflete que todo nó tem aresta com todos os outros, exceto com ele mesmo.

9. Aplicações em Redes e Software

- **Redes de Computadores (Topologia de Malha Total):** Representa uma rede onde cada computador está conectado diretamente a todos os outros. Alta tolerância a falhas, mas custo altíssimo.
- **Problema do Caixeiro Viajante:** Encontrar o menor caminho passando por todos os vértices em um grafo completo com pesos nas arestas.
- **Sistemas Multiprocessados:** Conexões entre processadores e memória.
- **Redes Sociais (Cliques):** Modelagem de subgrupos onde todos os membros interagem entre si.

10. Introdução aos Teoremas em Grafos Completos

- Grafos completos (K_n) são estruturas extremas na teoria dos grafos.
- Devido à sua densidade máxima de arestas, eles servem como casos base ou limites superiores para muitos teoremas.
- A seguir, exploraremos as principais propriedades estruturais, métricas de conectividade e comportamentos de travessia (Euler e Hamilton).
- Usaremos as definições de **Teorema** (proposição provada verdadeira) e **Corolário** (consequência direta de um teorema).

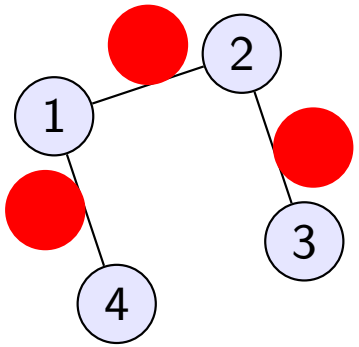
11. Teorema 1: O Lema do Aperto de Mãos

Teorema (Lema do Aperto de Mãos)

Em qualquer grafo não direcionado $G = (V, E)$, a soma dos graus de todos os vértices é igual ao dobro do número de arestas:

$$\sum_{v \in V} grau(v) = 2|E|$$

- **Explicação:** Cada aresta conecta exatamente dois vértices. Portanto, ao somar os graus de todos os vértices, cada aresta é contada exatamente duas vezes (uma para cada extremidade).
- Este teorema é universal, mas tem uma aplicação muito direta nos grafos completos.



Σ Graus $= 2 + 2 + 1 + 1 = 6$
$2 \times |E| = 2 \times 3 = 6$

12. Corolário 1: Número de Arestas em K_n

Corolário

Um grafo completo K_n possui exatamente $\frac{n(n-1)}{2}$ arestas.

- Como vimos anteriormente, essa propriedade deriva diretamente do Lema do Aperto de Mãos e do fato de K_n ser regular.

13. Prova do Corolário 1

Demonstração.

1. Seja K_n um grafo completo com n vértices.
2. Por definição, cada vértice em K_n é conectado a todos os outros $n-1$ vértices. Assim, $\text{grau}(v) = n-1$ para todo $v \in V$.
3. Aplicando o Lema do Aperto de Mãos:

$$\sum_{v \in V} \text{grau}(v) = 2|E|$$

4. Como existem n vértices, a soma é $n \times (n-1)$. Logo:

$$n(n-1) = 2|E| \implies |E| = \frac{n(n-1)}{2}$$

□

19. Propriedade 3: Ciclos Hamiltonianos

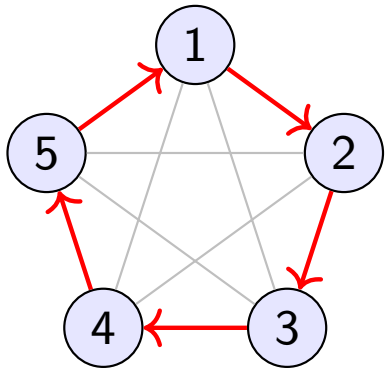
- **Ciclo Hamiltoniano:** Um ciclo que visita **cada vértice** de um grafo exatamente uma vez, voltando ao início.
- O Problema do Caixeiro Viajante busca o ciclo hamiltoniano de menor custo.
- Diferente do caminho Euleriano, encontrar um ciclo Hamiltoniano em um grafo genérico é um problema NP-Completo.
- **Mas e em grafos completos?**

20. Teorema 4: Ciclos Hamiltonianos em K_n

Teorema

Para todo $n \geq 3$, o grafo completo K_n possui ciclos Hamiltonianos.

- **Explicação:** Como existe uma aresta entre **qualquer** par de vértices, é trivial construir um ciclo: basta visitar os vértices em qualquer ordem de 1 a n, e sempre existirá a aresta $(i, i+1)$, bem como a aresta final $(n, 1)$.



Exemplo de Ciclo em K_5

21. Corolário 3: Número de Ciclos Hamiltonianos em K_n

Corolário

O número de ciclos Hamiltonianos distintos em um grafo completo K_n é $\frac{(n-1)!}{2}$.

Demonstração.

1. Existem $n!$ permutações possíveis para visitar os n vértices.
2. Como é um ciclo, o vértice inicial não importa (uma rota ABC é o mesmo ciclo que BCA e CAB), então dividimos por n, resultando em $(n-1)!$.
3. Como o grafo não é direcionado, percorrer o ciclo num sentido (A-B-C) ou no oposto (C-B-A) gera o mesmo conjunto de arestas, então dividimos por 2. Resulta em $\frac{(n-1)!}{2}$.

□

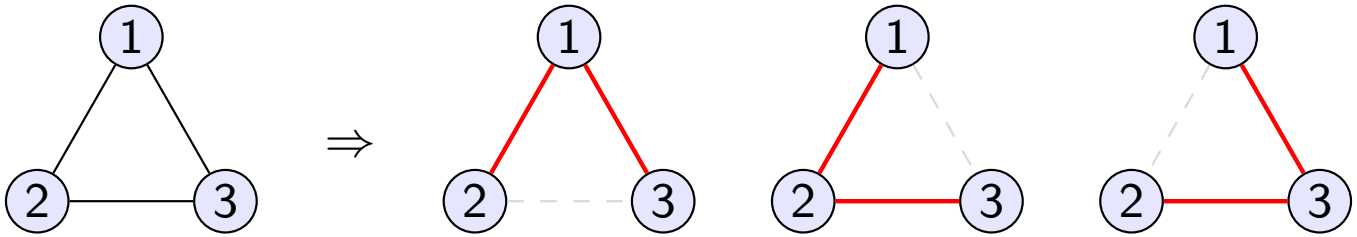
29. Teorema 8: Árvores Geradoras em K_n

- Uma **árvore geradora** de um grafo é um subgrafo que é uma árvore (sem ciclos) e que conecta todos os vértices do grafo original.

Teorema (Fórmula de Cayley)

O número de árvores geradoras distintas em um grafo completo rotulado K_n é dado por: n^{n-2}

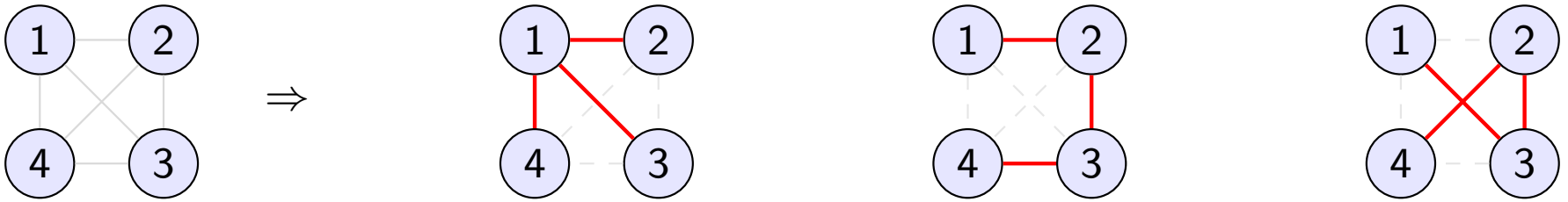
- Exemplo para K_3 (triângulo): O número de árvores geradoras é $3^{3-2} = 3^1 = 3$. (Basta remover uma das 3 arestas do triângulo).



K_3

30. Exemplos de Árvores Geradoras em K_4

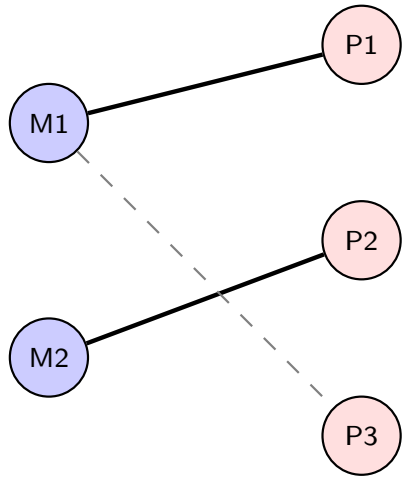
- Pela Fórmula de Cayley, K_4 possui $4^{4-2} = 16$ árvores geradoras distintas.
- Abaixo estão 3 exemplos dessas 16 possíveis árvores (lembre-se: elas devem conectar os 4 vértices sem formar ciclos):



K_4 (Grafo Base) | Exemplo 1 (Estrela) | Exemplo 2 (Caminho) | Exemplo 3 (Zig-Zag)

31. Introdução aos Grafos Bipartidos

- Até agora vimos os grafos completos, onde "todo mundo se conecta com todo mundo".
- Na vida real (e na computação), muitas vezes lidamos com **dois conjuntos distintos de entidades** que só interagem entre si, mas nunca dentro do próprio grupo.
- Exemplos: Motoristas e Passageiros (Uber), Alunos e Disciplinas, Clientes e Servidores, Trabalhadores e Tarefas.
- Para modelar isso, utilizamos a poderosa estrutura dos **Grafos Bipartidos**.



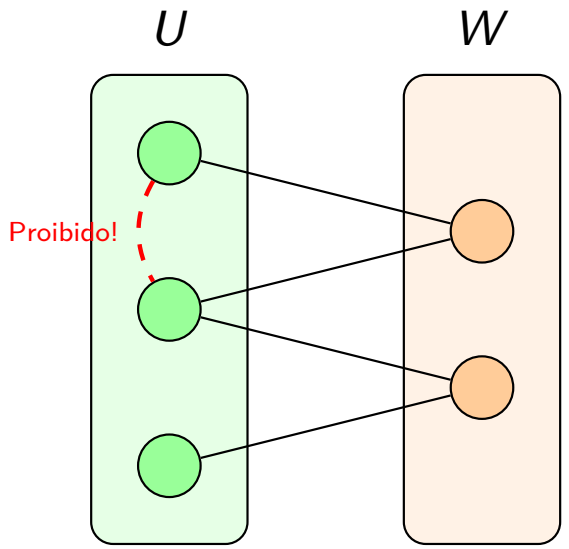
Motoristas (M) x Passageiros (P)

32. Definição Formal

Grafo Bipartido

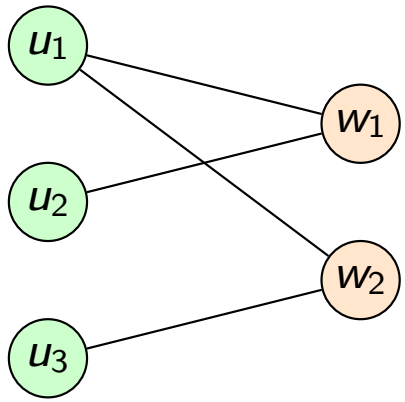
Um grafo $G = (V, E)$ é bipartido se o seu conjunto de vértices V pode ser particionado em dois subconjuntos disjuntos U e W (ou seja, $U \cup W = V$ e $U \cap W = \emptyset$), de tal forma que toda aresta em E conecte um vértice em U a um vértice em W.

- **Regra de Ouro:** Nenhuma aresta conecta dois vértices do mesmo conjunto.
- U e W são frequentemente chamados de "lados" ou "partições" do grafo.



33. Representação Visual

- Na representação visual clássica, colocamos os vértices do conjunto U à esquerda e os de W à direita.
- Todas as arestas "cruzam" o espaço entre os dois lados.



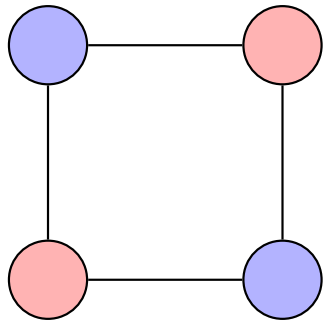
Conjunto U (Verde) e Conjunto W (Laranja)

34. Propriedade Fundamental: 2-Colorabilidade

Teorema

Um grafo é bipartido se e somente se o seu Número Cromático $\chi(G)$ for menor ou igual a 2.

- Isso significa que você sempre consegue pintar os vértices de um grafo bipartido usando apenas **duas cores**, sem que dois vértices da mesma cor fiquem conectados.
- Na prática: Pinte todos os vértices de U com a Cor 1 e todos de W com a Cor 2. Como não há arestas dentro de U nem dentro de W, a regra de coloração é respeitada perfeitamente.



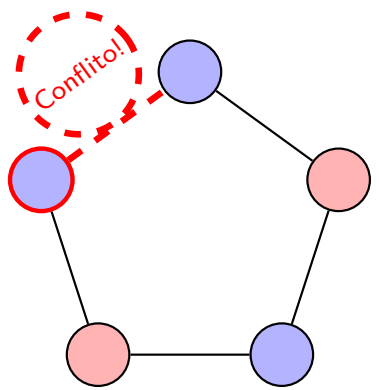
$\chi(G) = 2$ (Apenas Azul e Vermelho)

35. Condição Necessária: Ausência de Ciclos Ímpares

Teorema (Teorema de König)

Um grafo é bipartido se e somente se ele ***não possui nenhum ciclo de comprimento ímpar*** *(ciclos com 3, 5, 7... arestas).*

- Se houvesse um ciclo ímpar (ex: um triângulo), ao tentar pintar o grafo com 2 cores, você inevitavelmente chegaria em dois vértices adjacentes que precisariam ter a mesma cor.
- Assim que você encontra um ciclo ímpar (ex: subgrafo K_3) em qualquer rede, você tem certeza matemática de que ela **não** é bipartida.



Ciclo de 5 arestas (Falha ao colorir)

36. O Grafo Bipartido Completo ($K_{m,n}$)

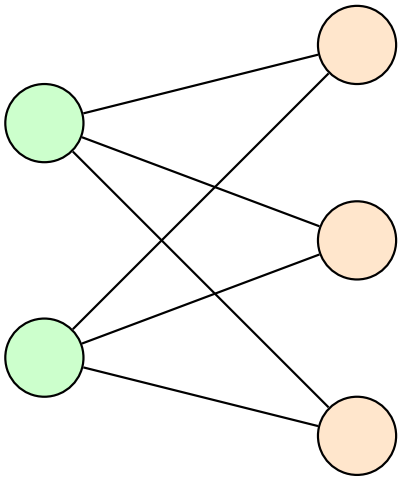
Definição

Um **Grafo Bipartido Completo**, denotado por $K_{m,n}$, é um grafo bipartido onde **cada** vértice do conjunto U está conectado a **todos** os vértices do conjunto W.

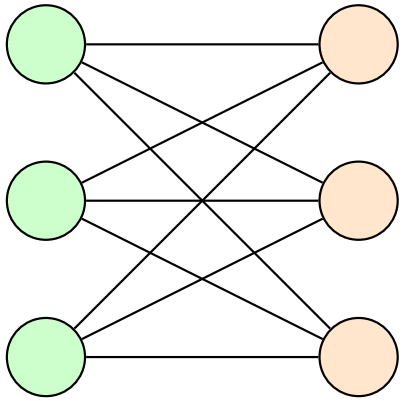
- Sendo $m = |U|$ (número de vértices em U) e $n = |W|$ (número de vértices em W).
- É a estrutura "mais densa" possível sem violar a regra de ser bipartido.
- Faz paralelo com o K_n, mas focado nas relações extremas entre dois grupos.

37. Exemplos de $K_{m,n}$: $K_{2,3}$ e $K_{3,3}$

$K_{2,3}$



$K_{3,3}$
(Crucial no Teorema da Planaridade!)



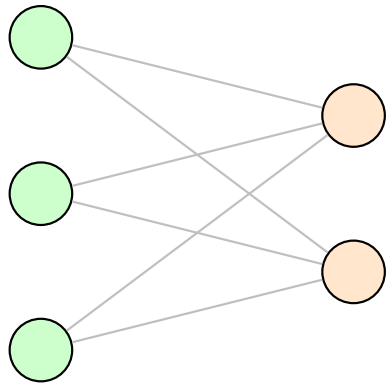
38. Propriedades de $K_{m,n}$

- **Número de Vértices:** O total de vértices em $K_{m,n}$ é simplesmente a soma $m + n$.
- **Número de Arestas:** Como cada vértice dos m se conecta a cada um dos n, o número total de arestas é dado pelo produto cartesiano das partições:

Fórmula do Número de Arestas em $K_{m,n}$

$$|E| = m \times n$$

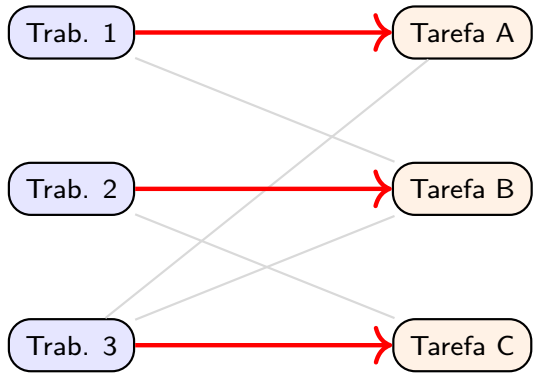
- Exemplo: $K_{3,3}$ possui $3 + 3 = 6$ vértices e $3 \times 3 = 9$ arestas.
- Todos os vértices do conjunto de tamanho m possuem grau n, e vice-versa.



$K_{3,2}$ com $3 \times 2 = 6$ arestas

39. Aplicação: O Problema da Atribuição (Matching)

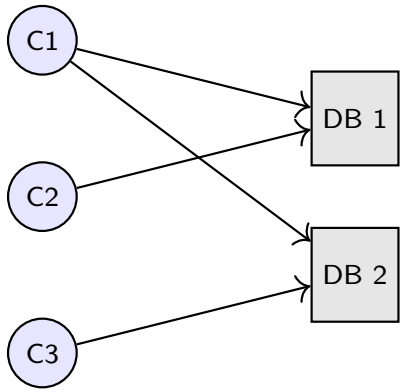
- **Problema Prático:** 3 trabalhadores e 3 tarefas. Como associar de forma ótima?
- **Solução Bipartida:** Conjunto U (Trabalhadores) e W (Tarefas). Arestas indicam "sabe fazer".
- Objetivo do **Algoritmo Húngaro**: Encontrar o **Casamento Perfeito** (cada trabalhador faz exatamente uma tarefa).



Arestas em vermelho (Casamento Perfeito)

40. Aplicação em Arquitetura de Software

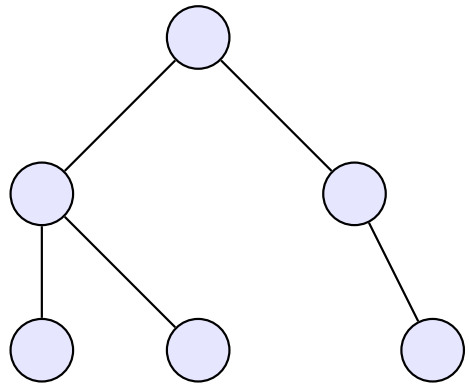
- **Sistemas Cliente-Servidor:** Clientes não interagem diretamente entre si, mas todos falam com os Servidores.
- **Bancos de Dados Relacionais:** Tabelas e chaves estrangeiras comumente formam relações bipartidas.
- **Modelagem:** Compreender a arquitetura de grafo bipartido economiza recursos: *você não precisa preparar o sistema para conexões entre clientes!*



C (Clientes) e *DB* (Servidores)

41. O que é uma Árvore (em Grafos)?

- Na teoria dos grafos, uma **Árvore** é um grafo conexo e **acíclico** (não possui ciclos).
- Se você adicionar apenas **uma aresta** a uma árvore, você cria um ciclo. Se remover uma aresta, você desconecta o grafo.
- Em Engenharia de Software, as árvores são usadas para modelar **hierarquia** (ex: sistemas de pastas do Windows/Linux, hierarquias de classes na Orientação a Objetos).



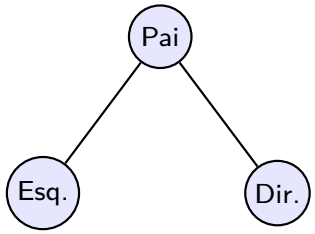
Exemplo de Grafo Acíclico Conexo (Árvore)

42. A Árvore Binária

Definição

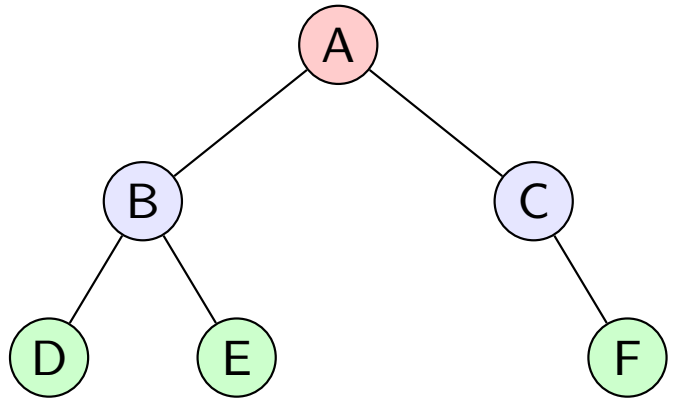
Uma **Árvore Binária** é uma árvore com raiz na qual cada nó tem **no máximo dois filhos**, tipicamente chamados de *filho esquerdo* e *filho direito*.

- Diferente dos grafos completos ou bipartidos, onde os nós têm muitos vizinhos "iguais", na árvore binária existe uma relação rígida de **pai-filho**.
- Isso a torna extremamente eficiente para decisões de "sim/não" ou "maior/menor".



43. Anatomia de uma Árvore Binária

- **Raiz (Root):** O topo da árvore (nível 0).
- **Nós Internos:** Possuem pelo menos 1 filho.
- **Folhas (Leaves):** Nós sem filhos (o fim da linha).
- **Altura (Height):** A maior distância da raiz até a folha mais profunda.



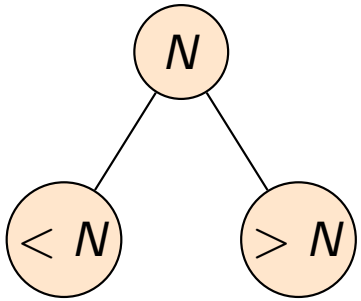
Raiz (Vermelho), Nós (Azul), Folhas (Verde)

44. Árvore Binária de Busca (BST)

- Uma das aplicações mais geniais das árvores binárias é a **Binary Search Tree (BST)**.
- Ela impõe uma **regra de ordenação lógica**:

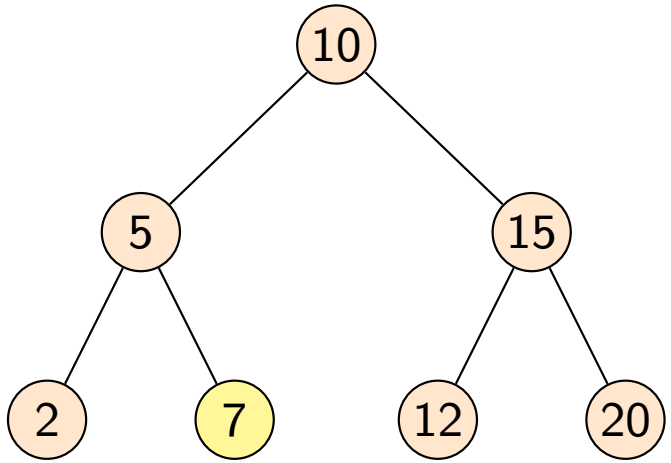
A Regra de Ouro da BST

Para todo nó N:
1. Todos os valores à **esquerda** de N são **menores** que o valor de N.
2. Todos os valores à **direita** de N são **maiores** que o valor de N.



45. Exemplo Visual de uma BST

- Veja como a busca funciona logicamente.
- Para buscar o número 7:
  1. $7 < 10$ (vai para a esquerda)
  2. $7 > 5$ (vai para a direita)
  3. $7 == 7$ (Encontrou!)
- Reduzimos as possibilidades pela metade a cada passo.

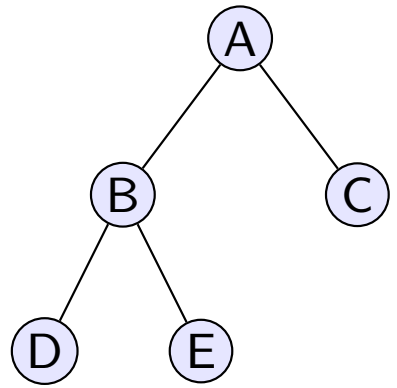


46. Classificação: Árvore Binária Estrita (Full)

Definição

Uma **Árvore Binária Estrita** (ou Cheia/Própria) é aquela em que **todo nó tem exatamente 0 ou 2 filhos**. Nenhum nó interno tem apenas 1 filho.

- Utilizada frequentemente para representar expressões matemáticas rigorosas (onde todo operador binário exige 2 operandos).



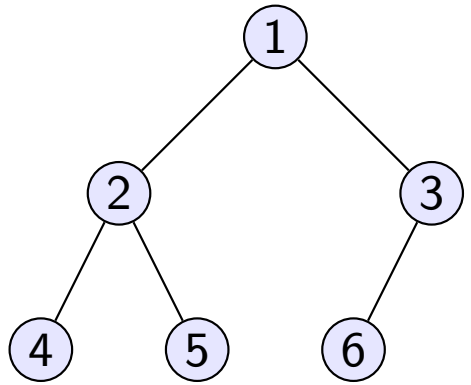
Árvore Estrita (0 ou 2 filhos)

47. Classificação: Árvore Binária Completa

Definição

Uma **Árvore Binária Completa** tem todos os seus níveis completamente preenchidos, *exceto possivelmente o último*, que deve ser preenchido estritamente **da esquerda para a direita**.

- Excelente para implementação utilizando **Arrays** em vez de ponteiros (a base da estrutura de dados *Heap*).



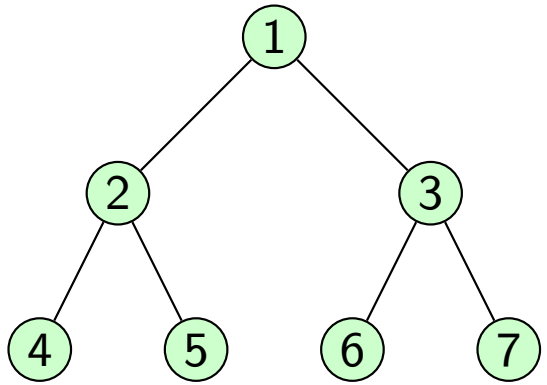
Árvore Completa (preenchida à esquerda)

48. Classificação: Árvore Binária Perfeita

Definição

Uma **Árvore Binária Perfeita** é aquela em que **todos os nós internos têm 2 filhos** e **todas as folhas estão exatamente no mesmo nível**.

- É a árvore binária mais densa possível para uma dada altura.
- Extremamente rara na prática, mas fundamental para a análise de complexidade (o melhor caso).



Árvore Perfeita (Totalmente Simétrica)

49. Teorema 1: Nós Máximos por Nível

Teorema

O número máximo de nós em um determinado nível k de uma árvore binária é 2^k, considerando a raiz no nível $k = 0$.

- **Nível 0 (Raiz):** $2^0 = 1$ nó.
- **Nível 1:** $2^1 = 2$ nós (filhos da raiz).
- **Nível 2:** $2^2 = 4$ nós.
- **Nível 3:** $2^3 = 8$ nós.
- Como a ramificação é binária (base 2), a cada nível a capacidade dobra. O crescimento é exponencial.

50. Teorema 2: Nós em uma Árvore Perfeita

Teorema

O número total de nós (N) de uma árvore binária perfeita de altura h é a soma de uma progressão geométrica: $N = 2^{h+1} - 1$.

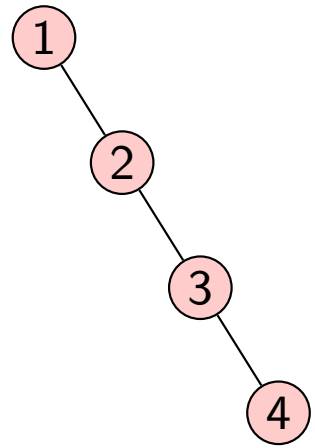
- Para uma árvore perfeita com altura $h = 2$ (Raiz + 2 níveis abaixo):
- $N = 2^{2+1} - 1 = 2^3 - 1 = 8 - 1 =$ **7** nós.
- Isso prova matematicamente o porquê de buscas binárias serem tão rápidas: com apenas altura 20 ($h = 20$), podemos armazenar $2^{21} - 1 \approx 2$ **milhões** de nós!

51. Teorema 3: Árvore Degenerada (Pior Caso)

Teorema

O número mínimo de nós (N) em uma árvore binária de altura h é $N = h + 1$.

- Isso acontece quando cada nó tem **apenas 1 filho**.
- A árvore binária sofre uma "degeneração"e se transforma em uma simples **Lista Encadeada** (*Linked List*).
- Toda a vantagem do $O(\log N)$ é perdida, voltando a ser uma busca ineficiente $O(N)$.



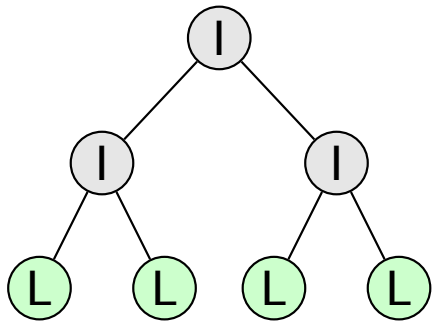
Árvore Degenerada (Pior caso)

52. Propriedade: Folhas vs. Nós Internos

Teorema

*Em qualquer **Árvore Binária Estrita** (onde nós têm 0 ou 2 filhos), o número de folhas (L) é sempre igual ao número de nós internos (I) mais um:* $L = I + 1$.

- Não importa a estrutura da árvore. Se todo nó ramifica para 2, a base será exatamente 1 unidade maior que os nós do meio.
- Exemplo prático: Torneios de eliminação (chaveamentos esportivos). Para haver 8 times na base (Folhas), são necessários 7 jogos (Nós Internos).



$I = 3$ e $L = 4$

53. Teorema das Arestas em Árvores

Teorema

Toda árvore (binária ou genérica) com N vértices possui exatamente $N - 1$ ***arestas****.*

- Consequência direta do Lema do Aperto de Mãos.
- Como a árvore é conectada e acíclica, para conectar o primeiro nó a um novo nó, você precisa de exatamente 1 aresta extra.
- Se houvesse menos que $N - 1$ arestas, o grafo estaria **desconectado**.
- Se houvesse N ou mais arestas, inevitavelmente se formaria um **ciclo**.

54. A Relação Logarítmica: $h \approx \log_2(N)$

Equação da Altura Mínima

A altura mínima h_{min} para acomodar N nós é: $h \approx \lceil \log_2(N+1) \rceil - 1$

- Na computação, a altura h de uma árvore representa o **número máximo de "ifs"ou "saltos"** necessários para encontrar uma informação.
- O crescimento logarítmico significa que o volume de dados pode crescer brutalmente, mas a altura da árvore cresce de forma ínfima.
- Essa é a prova matemática do porquê bancos de dados e sistemas de busca são viáveis em larga escala.

55. Conclusão: O Desafio do Balanceamento

- Estudamos os **Teoremas** e vimos os **Extremos**:
- **Melhor Caso (Árvore Perfeita):** A busca é $O(\log N)$.
- **Pior Caso (Árvore Degenerada):** A busca é $O(N)$.

O que os Cientistas da Computação Fizeram?

Inventaram Árvores de Busca **Auto-Balanceáveis** (Árvore AVL, Red-Black). Toda vez que um nó é inserido, algoritmos aplicam "Rotações"na árvore para garantir matematicamente que ela se mantenha o mais próxima possível de uma Árvore Perfeita, salvaguardando a performance $O(\log N)$.